Acceptance Testing: Building the Right Thing

SE 413/513 — Software Testing

Course Reading Material

Unit: Test Levels

© Dr. Kamrul Hasan, Grand Valley State University. All rights reserved.
For permissions or questions regarding the use of this material, contact hasanka@gvsu.edu.

Please cite [1] when using, adapting, or reproducing this material. The corresponding BibTeX entry is provided below:

```
@misc{hasan2026stesting,
author = {Hasan, Kamrul},
year = {2026},
title = {Software Testing},
institution = {Grand Valley State University},
url = {https://github.com/mdkamrulhasan/software-testing}
}
```

Contents

1 Learning Objectives **3**

2 Where We Are **3**

3 What Is Acceptance Testing? **3**
3.1 Definition . . . 3
3.2 Validation versus Verification . . . 4
3.3 Forms of Acceptance Testing . . . 4
3.4 Acceptance Testing versus System Testing . . . 4

4 User Stories and Acceptance Criteria **5**
4.1 User Stories . . . 5
4.2 Acceptance Criteria . . . 5

5 Specification by Example **6**
5.1 Rules Are Ambiguous; Examples Are Not . . . 6
5.2 The Three Amigos . . . 6

6 ATDD and BDD **6**
6.1 Acceptance Test-Driven Development . . . 6
6.2 Behavior-Driven Development . . . 7

7 Writing Scenarios in Gherkin **7**
7.1 The Language . . . 7
7.2 The Coupon Feature . . . 7
7.3 Declarative, Not Imperative . . . 8

8 Automating Scenarios with `pytest-bdd` **9**
8.1 Layout . . . 9
8.2 Step Definitions . . . 9
8.3 Running the Scenarios . . . 10
8.4 Writing the Scenario First . . . 11

9 The Defect Only Acceptance Testing Found **12**
9.1 The Developer's Tests . . . 12
9.2 Every Level, With and Without the Examples . . . 13
9.3 Whose Defect Is It? . . . 13

10 Operational Acceptance Testing **14**

11 From Accepted Criteria to an Accepted Release **14**
11.1 Done Means Accepted . . . 14
11.2 The Acceptance Decision . . . 14

12 Common Acceptance Failures **15**

13 Putting It Together: A Hands-On Walkthrough **15**

14 Best Practices **16**

15 Common Mistakes **16**

16 Applying This to Your Semester Project **17**

17 A Practical Mental Model **17**

18 Summary **18**

19 Practice Exercises **18**

20 Discussion Questions **19**

21 Key Takeaways for SE 413/513 **19**

22 References **20**

1 Learning Objectives

After completing this reading, you should be able to:

- Define acceptance testing, explain how **validation** differs from **verification**, and distinguish acceptance testing from system testing.
- Describe the main forms of acceptance testing — user, operational, contractual, regulatory, alpha, and beta — and give an example of each.
- Write user stories with **acceptance criteria** that are testable, and refine them with **concrete examples** agreed by the "three amigos".
- Explain **acceptance test-driven development (ATDD)** and **behavior-driven development (BDD)**, and how they relate to Week 4's TDD.
- Write acceptance scenarios in **Gherkin** (`Feature`, `Background`, `Scenario`, `Scenario Outline`, `Examples`, data tables) and automate them with `pytest-bdd`.
- Explain why a system can pass all of its developer, integration, and system tests and still fail acceptance, and give an example.

2 Where We Are

The companion reading, *System Testing*, ran the shop as a real server and tested it against its system requirements. It ended with a table:

	Unit	**Integration**	**System**	**Acceptance**
`SHOP_SERVICE=racy` (oversells)	pass	pass	**1 failed**	pass
`SHOP_COUPONS=per_unit`	pass	pass	pass	**2 failed**

The second row is this reading's subject. The shop with `SHOP_COUPONS=per_unit` passes all 8 of its developer's unit tests, all 13 integration tests, and all 23 system tests. Its coupon code works, consistently, exactly as its developer understood the story. It is still not what the customer asked for — and only the tests written *with* the customer, in the customer's words, notice.

Key Idea

Every test level so far checks the software against something the development team wrote: a docstring, a contract, a system specification. Acceptance testing checks it against what the *customer* needs. If the team misunderstood the need, every earlier level can pass.

3 What Is Acceptance Testing?

3.1 Definition

Acceptance testing is the test level that focuses on **validation**: on demonstrating that the system meets the users' business needs and is ready to be deployed[2]. Ideally it is performed by, or with, the intended users. Its test basis is the business view of the system: user stories and their acceptance criteria, business processes, use cases, contracts, regulations, and operational procedures. Its outcome is a *decision*: is this release acceptable?

3.2 Validation versus Verification

Barry Boehm's informal definitions are still the clearest[4]:

- **Verification:** "Am I building the product right?" — does the software match its specification? Unit, integration, and system testing are verification.
- **Validation:** "Am I building the right product?" — does the specification (and so the software) match what the users need? Acceptance testing is validation.

A defect found by verification is a mismatch between code and specification. A defect found by validation may be a mismatch between the specification and the need — or, as in this reading, between the need and *one person's reading* of a specification that could be read two ways.

3.3 Forms of Acceptance Testing

The ISTQB syllabus names five main forms[2]:

Form	**Who accepts, and what**	**For the shop**
User acceptance testing (UAT)	End users: can I do my job / reach my goal with it?	The product owner runs the coupon scenarios (Section 7)
Operational acceptance testing (OAT)	Operations staff: can we deploy, run, monitor, back up, and recover it?	Start, `/health`, restart with data intact (Section 10)
Contractual acceptance testing	The client, against criteria written in a contract	"Orders of up to 100 units, 10 customers at once, under 1 s"
Regulatory acceptance testing	An authority, against laws and standards	Card-payment security rules; accessibility law for a storefront
Alpha and beta testing	Real users, before general release: alpha at the developer's site, beta at the users' own	50 regular customers try coupons for a week before launch

Most of this reading is about the first form, because it is the one a development team can automate: the user's acceptance criteria, agreed in advance and turned into executable tests. Section 10 covers operational acceptance.

3.4 Acceptance Testing versus System Testing

Both test the whole system from the outside, and in this course both drive the same running server. They differ in the question, and so in the test basis, who writes the tests, and what a failure means:

	System testing	Acceptance testing
Question	Does it meet its specification?	Is it what the users need?
Test basis	System requirements (SYS, NFR)	User stories, acceptance criteria, business rules
Language	The team's: status codes, cents, `SYS-4`	The customer's: "I am charged $20.00"
Written by	Testers, often independent	Customer, developer, and tester together
Failure means	The system is wrong	The system, *or our understanding*, is wrong
In the demo	`tests/system/`: 23 tests	`tests/acceptance/`: 14 Gherkin scenarios

4 User Stories and Acceptance Criteria

4.1 User Stories

Agile teams usually capture requirements as **user stories**: short descriptions of a feature from the point of view of the person who wants it[5]:

> **As a** *role*, **I want to** *goal*, **so that** *benefit*.

A story is deliberately incomplete. Ron Jeffries described it as three parts: the **Card** (the one-sentence story), the **Conversation** (the discussion that fills in the details), and the **Confirmation** (the tests that show it is done)[6]. Acceptance tests are the third C. A widely used checklist for good stories is **INVEST**: Independent, Negotiable, Valuable, Estimable, Small, Testable[7]. The last letter is this course's concern: a story without a way to confirm it is not ready.

4.2 Acceptance Criteria

Acceptance criteria are the conditions a story must satisfy to be accepted. The shop's coupon story, from `specs/user_stories.md`:

US-3 Coupon codes

As a customer, **I want to** enter a coupon code when I order, **so that** I pay less.

US-3.1 `SAVE10` takes 10% off the order, rounded down to the cent.
US-3.2 `FLAT5` takes $5.00 off the **order**, once, however many units are in it.
US-3.3 `FLAT5` can only be used on an order of **$10.00 or more** (before the discount).
US-3.4 Codes are not case-sensitive: `save10` works.
US-3.5 An unknown code is rejected with a message, and I am not charged.
US-3.6 The confirmation shows the price before the discount, the discount, and what I paid.

Good acceptance criteria are written in the customer's language, are each testable on their own, say what happens in the unhappy cases (US-3.5), and avoid saying *how* (no database, no endpoint names). The two words in bold in US-3.2 and US-3.3 were not in the first draft. Section 5 shows where they came from.

Common Pitfall

Acceptance criteria that only restate the story. "Coupons work correctly" cannot fail. Neither can "FLAT5 gives $5 off" — until someone asks "off what?", which is the question the criterion exists to answer.

5 Specification by Example

5.1 Rules Are Ambiguous; Examples Are Not

The first draft of the coupon story said only: "`SAVE10` gives 10% off. `FLAT5` gives $5 off orders over $10." Every word of that is clear, and it can still be implemented in several different ways. Gojko Adzic's **specification by example** is the practice of resolving such rules by agreeing on **concrete examples** — specific inputs with specific expected outcomes — and using those examples, unchanged, as the acceptance tests[8].

5.2 The Three Amigos

The examples are found in a short meeting of three perspectives, often called the **three amigos**[9]: the **business** (the product owner, who decides what is wanted), **development** (who knows what is possible and what is ambiguous to implement), and **testing** (who asks "what about…?"). For US-3, the questions and answers were recorded in the story file:

Question	Product owner's answer	Example it became
Tester: Is FLAT5 $5 off each mug, or off the whole order?	The whole order.	2 mugs ($25.00) → pay $20.00
Tester: Is the $10 minimum one item's price, or the order's?	The order's.	4 stickers ($12.00) → pay $7.00
Developer: What if 10% is not a whole number of cents?	Round the discount down.	1 pen ($1.99) → discount $0.19
Developer: Can two coupons be combined?	Not this release.	(out of scope, recorded)

Notice who asked the two FLAT5 questions. A developer who already "knows" what the story means does not ask them. A tester, whose job is to imagine the cases nobody mentioned, does — the same instinct as filling in every column of a decision table in Week 8.

Key Idea

In functional testing (Week 8), the oracle is the specification. In acceptance testing, the oracle is the **customer**, and the agreed examples are how the customer's judgment is written down *before* the code exists, so that it can be checked automatically afterwards.

6 ATDD and BDD

6.1 Acceptance Test-Driven Development

In Week 4's TDD, a developer writes a failing *unit* test, then the code. **Acceptance test-driven development (ATDD)** applies the same idea one level up: the team writes the failing *acceptance*

tests, from the agreed examples, before the feature is built[10]. The two loops nest (Figure 1):

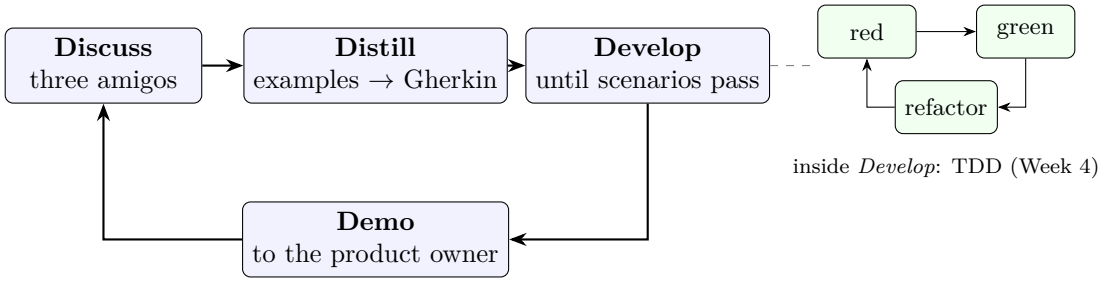


Figure 1: ATDD's outer loop around TDD's inner loop. The acceptance scenarios are written in *Distill*, fail until the end of *Develop*, and are shown passing in *Demo*.

The four steps are Ken Pugh's[10]. Inside *Develop*, the developer still works test-first with unit tests; the acceptance scenarios say when the whole feature is done. A failing acceptance test is the to-do list; a passing one is the confirmation.

6.2 Behavior-Driven Development

Behavior-driven development (BDD), introduced by Dan North, grew out of TDD with one change of emphasis: talk about *behavior* rather than tests, in a structured language the business can read[11]. Its lasting contribution is the scenario template:

> **Given** some context, **When** an event happens, **Then** expect an outcome.

This is the Arrange–Act–Assert of Week 2, in business language[12]. In practice ATDD and BDD describe the same workflow from different sides; the tools that support them read scenarios written in **Gherkin**.

7 Writing Scenarios in Gherkin

7.1 The Language

Gherkin is a small, line-oriented language for executable specifications, originally from the Cucumber tool[13]. A `.feature` file contains:

Keyword	Meaning
`Feature:`	The story, with its narrative ("As a…") as free text
`Background:`	Steps run before every scenario in the file (shared context)
`Scenario:`	One example: a sequence of steps
`Given / When / Then`	Context / action / expected outcome
`And / But`	Another step of the same kind as the one before
`Scenario Outline:`	A scenario template with `<placeholders>`…
`Examples:`	…and the table of values that fills them in, one run per row
`| a | b |`	A data table attached to a step
`#`	A comment (used here to trace scenarios to acceptance criteria)

7.2 The Coupon Feature

The examples from Section 5 become `tests/acceptance/features/coupons.feature`:

```
Feature: Coupon codes at checkout
  As a customer
  I want to enter a coupon code when I order
  So that I pay less

  Background:
    Given the shop sells:
      | sku     | name           | price | stock |
      | MUG     | Coffee mug     | 12.50 | 10    |
      | STICKER | Laptop sticker | 3.00  | 100   |
      | PEN     | Gel pen        | 1.99  | 50    |

  # US-3.1 to US-3.4, US-3.6
  Scenario Outline: A valid coupon reduces the amount charged
    When I order <quantity> "<sku>" with coupon "<code>"
    Then my order is confirmed
    And my confirmation shows a subtotal of $<subtotal>, a discount of $<discount>, and a total of
    $<total>
    And I am charged $<total>

    Examples:
      | sku     | quantity | code   | subtotal | discount | total |
      | MUG     | 1        | SAVE10 | 12.50    | 1.25     | 11.25 |
      | MUG     | 3        | SAVE10 | 37.50    | 3.75     | 33.75 |
      | PEN     | 1        | SAVE10 | 1.99     | 0.19     | 1.80  |
      | MUG     | 1        | FLAT5  | 12.50    | 5.00     | 7.50  |
      | MUG     | 2        | FLAT5  | 25.00    | 5.00     | 20.00 |
      | STICKER | 4        | FLAT5  | 12.00    | 5.00     | 7.00  |
      | MUG     | 1        | save10 | 12.50    | 1.25     | 11.25 |

  # US-3.3
  Scenario: FLAT5 needs an order of at least $10.00
    When I order 3 "STICKER" with coupon "FLAT5"
    Then I am told "FLAT5 requires an order of at least $10.00"
    And I am not charged
    And the catalog shows 100 "STICKER" in stock

  # US-3.5
  Scenario: An unknown coupon is rejected
    When I order 1 "MUG" with coupon "FREESTUFF"
    Then I am told "unknown coupon code: FREESTUFF"
    And I am not charged
```

Read it as the product owner would. There are no status codes, no cents, no endpoints: money is in dollars, the customer is "I", and every row of the `Examples` table is one of the agreed examples. A product owner can check this file line by line against the three-amigos notes — which is the point. The `Background` also makes each scenario independent: it creates the shop's catalog from the table, so the scenario does not depend on whatever `serve.py`'s default products happen to be.

7.3 Declarative, Not Imperative

A scenario should describe *what* the user does and observes, not *how* they do it through the interface:

```
# Imperative: brittle, and unreadable to the product owner
When I send POST /orders with header X-API-Key "shop-key" and JSON body
  {"sku": "MUG", "quantity": 2, "customer_email": "ana@example.com", "coupon": "FLAT5"}
```

```
Then the response status is 201 and total_cents is 2000

# Declarative: what the agreed example says
When I order 2 "MUG" with coupon "FLAT5"
Then my order is confirmed
And I am charged $20.00
```

The declarative version survives an API redesign unchanged; only the step definitions behind it change.

Common Pitfall

Gherkin written by developers, for developers. If the product owner cannot read a scenario and say "yes, that's right" or "no, that's wrong", it is a system test with extra syntax — and it has lost the one thing that made it an acceptance test.

8 Automating Scenarios with `pytest-bdd`

8.1 Layout

`pytest-bdd` is a `pytest` plugin that runs Gherkin scenarios as ordinary `pytest` tests, with ordinary fixtures[14]. It is in this week's `requirements.txt`. The demo's layout:

```
tests/acceptance/
    features/browse_products.feature   US-1
    features/place_order.feature       US-2
    features/coupons.feature           US-3
    conftest.py                        step definitions, shared by all three
    test_browse_products.py            scenarios("features/browse_products.feature
    ")
    test_place_order.py                ...
    test_coupons.py                    ...
```

Each test module is three lines — it only says which feature file to run:

```
import pytest
from pytest_bdd import scenarios

pytestmark = pytest.mark.acceptance

scenarios("features/coupons.feature")
```

8.2 Step Definitions

A **step definition** is a Python function that matches one kind of Gherkin line and does what it says. They live in `conftest.py` so that every feature can use every step. A few:

```
from decimal import Decimal
from pytest_bdd import given, parsers, then, when

def cents(dollars):
    """'12.50' -> 1250, exactly (no float rounding)."""
    return int(Decimal(dollars) * 100)
```

```python
@given("the shop sells:", target_fixture="the_shop")
def the_shop_sells(start_shop, datatable):
    header, *rows = datatable
    catalog = []
    for row in rows:
        product = dict(zip(header, row))
        catalog.append({"sku": product["sku"], "name": product["name"],
                        "price_cents": cents(product["price"]),
                        "stock": int(product["stock"])})
    return start_shop(seed=catalog)


@when(parsers.parse('I order {quantity:d} "{sku:w}" with coupon "{code:w}"'),
      target_fixture="response")
def i_order_with_coupon(the_shop, quantity, sku, code):
    return the_shop.order(sku, quantity, coupon=code)


@then(parsers.parse("I am charged ${amount}"))
def i_am_charged(the_shop, amount):
    assert [c["amount"] for c in the_shop.charges()] == [cents(amount)]


@then("I am not charged")
def i_am_not_charged(the_shop):
    assert the_shop.charges() == []
```

Four mechanisms do the work:

- **Parsers.** `parsers.parse` turns `{quantity:d}` into an `int` argument and `{sku:w}` into a one-word string.
- **`target_fixture`** makes a step's return value available as a fixture to later steps: the `Given` creates `the_shop`; the `When` creates `response`.
- **`datatable`** receives the table under a step as a list of rows, header first.
- **Ordinary fixtures.** `start_shop` is the *system test harness* from the companion reading. Acceptance scenarios drive the same running server, over HTTP, as the system tests: they differ in what they check and who agreed it, not in how they reach the system.

"I am charged $20.00" is checked against what PayFast says it charged (through the fake's test-only `/_test/charges`), not against the shop's own confirmation. The customer cares about the money that left their account; the confirmation is checked separately by its own step.

8.3 Running the Scenarios

```
$ pytest tests/acceptance -v
test_browse_products.py::test_the_catalog_shows_price_and_stock PASSED
test_browse_products.py::test_buying_reduces_the_stock_shown PASSED
test_coupons.py::test_a_valid_coupon_reduces_the_amount_charged[MUG-1-SAVE10
    -12.50-1.25-11.25] PASSED
...
```

```
test_coupons.py::test_a_valid_coupon_reduces_the_amount_charged[MUG-2-FLAT5
    -25.00-5.00-20.00] PASSED
test_coupons.py::test_a_valid_coupon_reduces_the_amount_charged[STICKER-4-FLAT5
    -12.00-5.00-7.00] PASSED
...
test_place_order.py::test_a_declined_card PASSED
14 passed in 3.35s
```

Each scenario becomes a test named after it; each `Examples` row becomes a parametrized case, identified by its values. With `--gherkin-terminal-reporter`, `pytest-bdd` prints the scenarios themselves as they run — a report the product owner can read:

```
$ pytest tests/acceptance/test_place_order.py --gherkin-terminal-reporter -vv
Feature: Place an order
    Scenario: A declined card
        Given the shop sells:
        Given my card will be declined
        When I order 2 "MUG"
        Then I am told "payment declined"
        And I am not charged
        And the catalog shows 5 "MUG" in stock
    PASSED
```

This is what is meant by **living documentation**: a specification that is checked every time the suite runs, so it cannot silently go out of date.

8.4 Writing the Scenario First

In ATDD the scenario exists before the code that makes it pass, and often before its step definitions. Suppose the product owner later decides that combining two coupons must be refused, and the team adds:

```
  Scenario: Two coupons cannot be combined
    When I order 2 "MUG" with coupons "SAVE10" and "FLAT5"
    Then I am told "only one coupon per order"
```

No step matches the `When` line yet, and `pytest-bdd` says so, naming the step, the line, and the scenario:

```
E   pytest_bdd.exceptions.StepDefinitionNotFoundError: Step definition is not
    found:
     When "I order 2 "MUG" with coupons "SAVE10" and "FLAT5"". Line 45 in scenario
     "Two coupons cannot be combined" in the feature ".../features/coupons.feature"
```

That is the ATDD "red": the next step is to write the step definition, then the feature.

Common Pitfall

Step patterns that match too much. The demo's first version of the plain ordering step was `parsers.parse('I order {quantity:d} "{sku}"')`. A bare `{sku}` matches *any* text, quotes included — so the new line above did not fail as undefined. It matched the old step with `sku` set to `MUG" with coupons "SAVE10" and "FLAT5`, ordered a product of that name, and failed two steps later with a confusing message:

```
E   assert {'error': 'unknown sku: MUG" with coupons "SAVE10" and "FLAT5'}
        == {'error': 'only one coupon per order'}
```

Constrain every parameter that is not the last thing on the line: `{sku:w}` (one word), `{quantity:d}` (an integer), or a regular expression with `parsers.re`.

9 The Defect Only Acceptance Testing Found

9.1 The Developer's Tests

Before the three-amigos session, the developer implemented the first draft of US-3 test-first, as Week 4 taught, with these unit tests (`tests/unit/test_pricing.py`, excerpt):

```python
def test_save10_takes_ten_percent():
    assert price_order(1250, 1, "SAVE10") == (1250, 125, 1125)

def test_flat5_takes_five_dollars():
    assert price_order(1250, 1, "FLAT5") == (1250, 500, 750)

def test_flat5_needs_ten_dollars():
    with pytest.raises(InvalidCoupon, match="at least"):
        price_order(300, 1, "FLAT5")

def test_quantity_multiplies_the_subtotal():
    subtotal, _, _ = price_order(1250, 3)
    assert subtotal == 3750
```

They are reasonable tests. They test rounding, case, an unknown code, the minimum. The implementation they drove, `shop/pricing_per_unit.py`, prices the order one unit at a time:

```python
def price_order(unit_price_cents, quantity, coupon=None):
    discount_per_unit = 0
    if coupon is not None:
        code = coupon.upper()
        if code == "SAVE10":
            discount_per_unit = unit_price_cents // 10
        elif code == "FLAT5":
            if unit_price_cents < FLAT5_MINIMUM_CENTS:    # one unit's price
                raise InvalidCoupon("FLAT5 requires an order of at least $10.00")
            discount_per_unit = 500                       # off each unit
        else:
            raise InvalidCoupon(f"unknown coupon code: {coupon}")
    subtotal = unit_price_cents * quantity
    discount = discount_per_unit * quantity
    return subtotal, discount, subtotal - discount
```

It is not careless code. It is a consistent implementation of "`FLAT5` gives $5 off orders over $10" under one reasonable reading — and for a single unit, the only quantity the developer's tests used, both readings agree.

9.2 Every Level, With and Without the Examples

```
$ SHOP_COUPONS=per_unit pytest tests/unit tests/integration -q
21 passed in 0.45s
$ SHOP_COUPONS=per_unit pytest tests/system -q
23 passed in 6.48s
$ SHOP_COUPONS=per_unit pytest tests/acceptance -q --tb=line
E   AssertionError: assert ('25.00', '10.00', '15.00') == ('25.00', '5.00', '20.00
    ')
E   AssertionError: {'error': 'FLAT5 requires an order of at least $10.00'}
FAILED ...test_a_valid_coupon_reduces_the_amount_charged[MUG-2-FLAT5
    -25.00-5.00-20.00]
FAILED ...test_a_valid_coupon_reduces_the_amount_charged[STICKER-4-FLAT5
    -12.00-5.00-7.00]
2 failed, 12 passed in 3.29s
```

The two failures are exactly the two examples that came from the tester's two questions. Two mugs with `FLAT5`: the per-unit version takes $10.00 off, not $5.00. Four stickers: $12.00 qualifies as an order, but one $3.00 sticker does not, so the per-unit version refuses the coupon. In the product owner's terms, through the Gherkin reporter:

```
Feature: Coupon codes at checkout
    Scenario Outline: A valid coupon reduces the amount charged
        Given the shop sells:
        When I order 2 "MUG" with coupon "FLAT5"
        Then my order is confirmed
        And my confirmation shows a subtotal of $25.00, a discount of $5.00, and a
     total of $20.00
        And I am charged $20.00
    FAILED
E   AssertionError: assert ('25.00', '10.00', '15.00') == ('25.00', '5.00', '20.00
    ')
```

9.3 Whose Defect Is It?

Not the developer's alone. The developer verified their own reading of the story, thoroughly. The story allowed that reading. The *examples* — agreed with the product owner, written down in Gherkin, and run against the real system — are what turned an ambiguous rule into a checkable one. Had the team skipped the three-amigos session, the first person to notice would have been a customer who bought two mugs and paid $15.00 — or the finance team, at the end of the month.

Key Idea

Developer tests check that the code does what the developer *thinks* the story means. Acceptance tests check what the *customer* meant. When those differ, only the second kind can notice, because only the second kind was written with the customer.

10 Operational Acceptance Testing

Customers are not the only people who must accept a release. **Operational acceptance testing (OAT)** asks whether the people who will *run* the system can do so: install and deploy it, start and stop it, monitor it, back it up and restore it, recover from failures, and roll back a bad release[2]. Many of these checks are system tests already; what OAT adds is the operators' acceptance criteria and their sign-off. For the shop:

Operational criterion	Evidence
It starts, and says when it is ready	`serve.py` prints `READY`; the harness relies on it
A load balancer can probe it without credentials	`test_health_needs_no_key`
A PayFast outage does not take it out of service	`test_health_stays_ok_during_a_provider_outage`
A restart loses no orders and does not reset stock	`test_orders_and_stock_survive_a_restart`
It recovers from a PayFast outage without a restart	`test_orders_work_again_after_a_provider_outage`
Every server in the pool runs the same version	*not yet checked* — see the case study

Real-World Case Study

Knight Capital, 2012. On August 1, 2012, a U.S. market-making firm's order router sent millions of unintended orders into the market in about 45 minutes, and the firm lost more than $460 million[15]. According to the U.S. Securities and Exchange Commission's findings, new code had been deployed to seven of the router's eight servers; a technician had not copied it to the eighth. The new code reused a flag that had once activated an old, unused feature. On the eighth server that old code was still present, and the flag woke it up. The SEC found that the firm had no written procedure requiring a second person to review the deployment. Every component might have passed its own tests; what was never accepted was the *deployment* — the system as the operators had actually put it into production.

11 From Accepted Criteria to an Accepted Release

11.1 Done Means Accepted

Many agile teams make passing acceptance tests part of the **Definition of Done**: a story is done when its scenarios pass against the integrated system, in the pipeline, and the product owner has seen them pass. The traceability is built in: every acceptance criterion in `user_stories.md` names the scenario that confirms it, and the scenario's comment names the criterion.

11.2 The Acceptance Decision

Automated scenarios inform the decision; they do not make it. The release decision also weighs things no scenario checks: whether the product owner, trying the feature, finds it usable; what alpha or beta users reported; open findings from system testing and their severity; and contractual or regulatory conditions. A release can pass every scenario and still be rejected — because the team

automated the wrong examples, or because the business changed its mind. Both are cheaper to discover in acceptance testing than after release.

Tip

Keep the acceptance suite small and meaningful. It is the most readable layer of the test pyramid, not the broadest: one scenario per agreed example, with the combinations and edge cases left to unit, functional, and system tests below it.

12 Common Acceptance Failures

Failure	Example	What would have caught it
Ambiguous rule, one reading implemented	FLAT5 per unit instead of per order	a three-amigos example with quantity 2
Rule stated, boundary not	the $10 minimum: one item or the order?	an example just above the minimum, made of cheap items
Unhappy path never agreed	what the customer sees for an unknown code	an acceptance criterion for each rejection
Rounding left to the developer	10% of $1.99	an example with an odd number of cents
Customer-visible outcome unchecked	the confirmation right, the charge wrong	a step that checks what was actually charged
Not operable	a release that cannot be deployed consistently	operational acceptance criteria and checks

13 Putting It Together: A Hands-On Walkthrough

To add a feature acceptance-test-first, work through five steps. As an example, take a new story:

> **US-4** As a returning customer, I want to see my past orders, so that I can check what I paid.

Step 1 — Draft the acceptance criteria. "I see my orders, newest first, with what I paid for each."

Step 2 — Hold a three-amigos session. Questions a tester should bring: *Which* orders — including declined ones? How do I prove they are mine (an email is not a password)? What do I see if I have none? Each answer becomes an example; the second may become a *security* requirement for the system test plan as well.

Step 3 — Distill the examples into Gherkin, in the customer's words:

```
Scenario: I see my paid orders, newest first
  Given I have ordered 1 "MUG" and then 4 "STICKER"
  When I look at my order history
  Then I see:
    | order | product | quantity | paid  |
    | 2     | STICKER | 4        | 12.00 |
    | 1     | MUG     | 1        | 12.50 |
```

Step 4 — Run it red, with `StepDefinitionNotFoundError`; write the step definitions (reusing the existing ones where they fit); run it red again, now for the right reason — the feature is missing.

Step 5 — Develop and demo. Build the feature test-first underneath (unit, integration, and system tests as usual). When the scenarios pass, show them — with `--gherkin-terminal-reporter` — to the product owner. Exercise 6 asks you to do this.

14 Best Practices

1. **Agree on examples before writing code,** with business, development, and testing in the room.
2. **Look for the examples that split the room:** quantities above one, values at boundaries, odd cents, the unhappy paths.
3. **Write scenarios in the customer's language** — dollars, "I", "my card" — and declaratively.
4. **One scenario per example, traced to its criterion** by a comment or a tag.
5. **Check what the customer experiences,** not only what the system reports about itself.
6. **Constrain step parameters** so a new line cannot silently match an old step.
7. **Run acceptance scenarios against the real, running system,** in the pipeline, and treat a failure as a conversation, not just a bug.
8. **Include the operators' acceptance criteria** in the release decision.

15 Common Mistakes

Common Pitfall

Writing acceptance tests after the code, from the code. They then confirm what was built, not what was needed — exactly what Week 8 warned against for expected values.

Common Pitfall

Skipping the conversation. Gherkin is not the point; the agreed examples are. A feature file written by one developer alone has the same blind spots as that developer's unit tests.

Common Pitfall

Imperative scenarios. Endpoints, status codes, and JSON in a `.feature` file make it unreadable to the business and brittle to change.

Common Pitfall

Step patterns that match too much. Constrain parameters (`{sku:w}`, `{quantity:d}`), or new steps will silently run old code.

Common Pitfall

Treating acceptance tests as the whole test suite. The racy service passes all 14 scenarios. Acceptance tests sit on top of the other levels, not instead of them.

Common Pitfall

Forgetting the operators. A system the customers accept and the operators cannot run safely is not ready.

16 Applying This to Your Semester Project

- The project guidelines plan acceptance tests for your user stories in Week 11. For each story, write its acceptance criteria and at least one concrete example per criterion.
- Hold a short three-amigos session: ask a classmate (or your instructor) to play the product owner. Record the questions and the answers, as in `specs/user_stories.md`.
- Write the examples as Gherkin in `tests/acceptance/features/`, automate them with `pytest-bdd`, and mark them `acceptance`. Reuse your system test harness to run them against the running system.
- In your report, show the traceability: story → criterion → example → scenario → result. If an acceptance test found a misunderstanding, say what it was and how the examples resolved it.
- Add two or three operational acceptance criteria (start-up, recovery, health) and say how each is checked.
- For Track A (existing open-source) projects: write the user stories from the project's documentation and issue tracker; the maintainers' stated intentions are your product owner.

17 A Practical Mental Model

When you read a story's acceptance criteria, think:

> "What concrete example would make two reasonable people give different answers?"

When you write a scenario, think:

> "Could the product owner read this aloud and say yes or no?"

When every lower-level test passes, think:

> "That proves we built it the way we understood it. Who checked that we understood it?"

18 Summary

Term	Meaning
Acceptance testing	Validating that the system meets the users' needs and is ready to deploy
Validation / verification	"Right product?" / "Product right?"
UAT, OAT	Acceptance by end users; by the people who will operate the system
Alpha / beta testing	Acceptance by real users at the developer's site / their own
User story	"As a ..., I want ..., so that ...": Card, Conversation, Confirmation
Acceptance criteria	The conditions a story must meet to be accepted
Specification by example	Resolving requirements with agreed, concrete examples that become tests
Three amigos	Business, development, and testing, agreeing the examples together
ATDD	Writing acceptance tests from the examples before building the feature
BDD / Gherkin	Behavior described as Given/When/Then; the language of `.feature` files
Step definition	The Python function that executes one kind of Gherkin step
Living documentation	A specification that is executed, so it cannot silently go stale

Key Idea

Agree on examples with the customer before you build; run them against the real system after. Acceptance testing is the only test level whose oracle is the person who wanted the software.

19 Practice Exercises

Exercise 1 — Find the Ambiguity

For the story "As a member, I want free shipping on large orders, so that I save money" (recall Week 8's shipping rules), write three questions a tester should ask in a three-amigos session, and the concrete example each answer would become.

Exercise 2 — Two Coupons

Implement the "Two coupons cannot be combined" scenario from Section 8 test-first: add the scenario, watch it fail with `StepDefinitionNotFoundError`, write the step definition, watch it fail for the right reason, then change the API so that it passes. What did you have to decide that the scenario did not say?

Exercise 3 — Declarative Rewrite

Rewrite this scenario declaratively, and write the step definitions it needs:

```
Scenario: Out of stock
  When I send POST /orders with {"sku": "TEE", "quantity": 1, "customer_email": "a@b.c"}
  Then the status code is 409
  And GET /_test/charges returns []
```

Exercise 4 — A New Coupon

The product owner wants `BULK20`: 20% off any order of 10 or more units of one product. Hold a three-amigos session with two classmates (one plays the product owner). Record the questions and answers, write the `Examples` table, and only then implement it.

Exercise 5 — Operational Criteria

The last row of the OAT table in Section 10 is unchecked. Propose an operational acceptance criterion and an automated check that would have caught Knight Capital's eighth server. (Hint: what could `/health` report?)

Exercise 6 — Order History

Carry US-4 from Section 13 through all five steps. Your three-amigos notes, the `.feature` file, and the failing run (before the feature exists) are part of the answer.

20 Discussion Questions

1. Explain the difference between verification and validation, using the per-unit coupon defect.
2. The developer's unit tests for `price_order()` were written test-first and all pass. What did TDD guarantee here, and what could it not?
3. Why is a tester, rather than a developer, more likely to ask "$5 off each mug, or off the order?"
4. Acceptance and system tests in this course drive the same running server. What, then, makes a test an acceptance test?
5. Why does the "I am charged" step check PayFast's record rather than the shop's confirmation?
6. When is Gherkin worth its cost, and when would plain `pytest` tests named after the acceptance criteria be enough?
7. Which forms of acceptance testing can be automated, and which cannot? Why?
8. The racy service passes every acceptance scenario. Is that a weakness of the acceptance suite? Should an acceptance scenario cover concurrency?

21 Key Takeaways for SE 413/513

1. **Acceptance testing is validation:** it checks the system against what users need, not against what the team wrote down.
2. **It has several forms:** user, operational, contractual, regulatory, alpha, and beta — and operators are users too.

3. **Acceptance criteria need examples;** the three amigos find the examples that resolve ambiguity.
4. **ATDD/BDD write the acceptance tests first,** in Gherkin, as living documentation the business can read.
5. **pytest-bdd runs Gherkin as pytest,** with step definitions, parsers, `target_fixture`, and the same harness as the system tests.
6. **A system can pass every other level and still be wrong:** the per-unit coupon passed 44 tests and failed 2 scenarios.
7. **Acceptance tests complement, not replace,** the levels below them.

22 References

- Test levels and forms of acceptance testing.[2, 3]
- Verification and validation.[4]
- User stories, Card–Conversation–Confirmation, and INVEST.[5, 6, 7]
- Specification by example and the three amigos.[8, 9]
- ATDD and BDD.[10, 11, 12]
- Gherkin and `pytest-bdd`.[13, 14]
- The Knight Capital incident.[15]
- Using `pytest`, this course's default testing framework.[16]
- Course material on software testing principles and related testing practices.[1]

References

[1] Hasan, K. (2026). *Software Testing*. Grand Valley State University. https://github.com/mdkamrulhasan/software-testing

[2] International Software Testing Qualifications Board. (2024). *Certified Tester Foundation Level (CTFL) Syllabus*, v4.0.1.

[3] Jorgensen, P. C. (2013). *Software Testing: A Craftsman's Approach* (4th ed.). CRC Press.

[4] Boehm, B. W. (1984). Verifying and validating software requirements and design specifications. *IEEE Software*, 1(1), 75–88. https://doi.org/10.1109/MS.1984.233702

[5] Cohn, M. (2004). *User Stories Applied: For Agile Software Development*. Addison-Wesley.

[6] Jeffries, R. (2001). Essential XP: Card, conversation, confirmation. *XProgramming.com*. https://ronjeffries.com/xprog/articles/expcardconversationconfirmation/

[7] Wake, B. (2003). INVEST in good stories, and SMART tasks. *XP123*. https://xp123.com/articles/invest-in-good-stories-and-smart-tasks/

[8] Adzic, G. (2011). *Specification by Example: How Successful Teams Deliver the Right Software*. Manning Publications.

[9] Dinwiddie, G. (2011). The three amigos: All for one, one for all. *Better Software*, 13(6), 24–27.

[10] Pugh, K. (2011). *Lean-Agile Acceptance Test-Driven Development: Better Software Through Collaboration*. Addison-Wesley.

[11] North, D. (2006). Introducing BDD. *Better Software*, March 2006. https://dannorth.net/introducing-bdd/

[12] Fowler, M. (2013). GivenWhenThen. https://martinfowler.com/bliki/GivenWhenThen.html

[13] Cucumber contributors. (2026). *Gherkin Reference*. https://cucumber.io/docs/gherkin/reference/

[14] pytest-bdd contributors. (2026). *pytest-bdd Documentation*. https://pytest-bdd.readthedocs.io/

[15] U.S. Securities and Exchange Commission. (2013). *In the Matter of Knight Capital Americas LLC*, Release No. 34-70694. https://www.sec.gov/litigation/admin/2013/34-70694.pdf

[16] Okken, B. (2022). *Python Testing with pytest* (2nd ed.). Pragmatic Bookshelf.